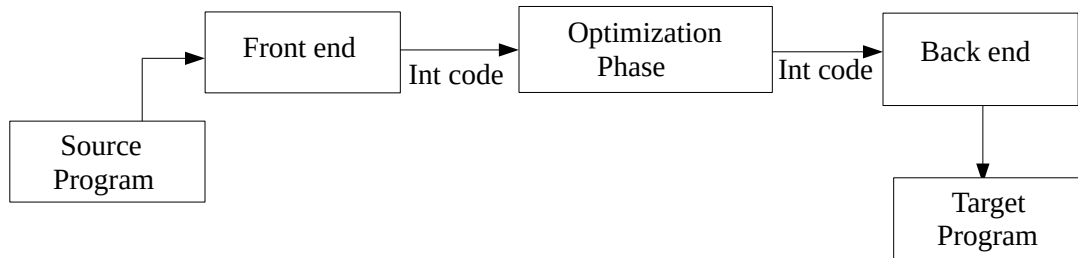


Introduction to Intermediate Code Optimization

Intermediate code (IC) optimization is an optional phase in most compilers which is enabled by a user, if so desired. For example in gcc, optimization can be enabled using the Optimization switch “-Ox”, where O denotes optimization, where x denotes a level, which is an integer, can be between 0 to 3. Level 0 denotes indicates no optimization, while levels 1, 2 and 3 specify optimizations. Level 2 is the optimization level most commonly deployed. Level 3 is used for architectures with advanced features, such as vector processors, multiple processors, etc.



As shown in the figure above, the main objective of the optimization phase is to improve the quality of intermediate code generated by the front end of the compiler.

We have already observed during the SDTS of various linguistic features, that the intermediate code generation concentrates on semantic correctness and equivalence and not on efficient code. Therefore the generated IC during SDTS often has redundant code fragments that can be eliminated or rewritten through analysis of the IC. For example,

- use of large number of temporaries to generate code to evaluate an expression; no concern for reusing temporaries for already evaluated expression values,
- generation of large number of “goto ____” during partial evaluation of conditional expressions in translation of control flow structures,
- multiple references to the same multiple dimensional array references, etc.

1. Processing of Intermediate Code to extract control flow information

The SDTS for all the features considered by us, except for declaration processing, results in generation of intermediate code. As illustrated in the figure above, the optimization phase of a compiler, which is optional, transforms the naive intermediate code to more efficient intermediate code by a sequence of transformations.

Before the transformation process starts, the first task in the optimization phase is to extract control flow information present in the intermediate code. The result is a directed graph, known as control flow graph.

Consider the following source code and its gimple intermediate code generated by gcc.

C Source program	Gimple intermediate code by gcc – edited to save space
<pre> int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; for (i = 1; i < 11; i++) { d = a * 15; if (i < 5) { x[i] = x[i-1] + d; b = a*c;} else { b = a * c; d = a * c; c = b + d;} }; printf(" a = %d b = %d d= %d x[5] = %d \n", a, b, d, x[5]); return 0; } </pre>	<pre> main () { int D.2329; { int a; int b; int c; int d; int i; int j; int x[10]; a = 2; b = 3; c = 40; x[0] = 10; x[1] = 20; x[2] = 30; x[3] = 40; x[4] = 50; x[5] = 60; x[6] = 70; x[7] = 80; x[8] = 90; x[9] = 100; i = 1; goto <D.2323>; <D.2322>: d = a * 15; if (i <= 4) goto <D.2326>; else goto <D.2327>; <D.2326>: _1 = i + -1; _2 = x[_1]; _3 = d + _2; x[i] = _3; b = a * c; goto <D.2328>; <D.2327>: b = a * c; d = a * c; c = b + d; <D.2328>: i = i + 1; <D.2323>: if (i <= 10) goto <D.2322>; else goto <D.2324>; <D.2324>: _4 = x[5]; printf (" a = %d b = %d d= %d x[5] = %d \n", a, b, d, _4); D.2329 = 0; return D.2329; } } } </pre>

Note that the intermediate code generated by us through our SDTS is similar to gimple in spirit. There are a few major differences :

- Temporaries generated by us are labeled as, ti, while gimple generally uses, _i, instead ; also gimple sometimes uses a temporary variable, such as D.2329 also.
- We label each intermediate code with numbers increasing sequentially, such as 10,11,...; while gimple used alphanumeric labels, such as <D.dddd>. Further, not every gimple statement is labeled, only a few selected are.
- Control flow instructions have a slightly different structure in our intermediate code format and that of gimple. Note that semantically they are equivalent.

Control flow	Intermediate code form used in the course	Gimple
conditional	if a relop b goto label 1 goto label 2	if (a relop b) goto label 1 else goto label 2
unconditional	goto label	goto label

- For function call, such as `printf(" a = %d b = %d d= %d x[5] = %d \n", a , b , d , x[5]);` gimple does very minimal change to the function call in the source, while we generate intermediate code that is fairly close to the structure of assembly code. The differences are explicated in the table below.

C function call : <code>printf(" a = %d b = %d d= %d x[5] = %d \n", a , b , d , x[5]);</code>	
Gimple code	Intermediate Code structure used by us
<code>_4 = x[5]; printf (" a = %d b = %d d= %d x[5] = %d \n", a, b, d, _4);</code>	<code>t1 := x[5] param " a = %d b = %d d= %d x[5] = %d \n" param a param b param d param t1 call printf</code>

Instead of our intermediate code, we shall use gimple code to construct a control flow graph (cfg) from it. The compiler gcc also constructs a control flow graph, which is usually saved as “programe.c.012t.cfg” for the source program, “programe.c”. For our program, the control flow graph constructed by gcc is shown in the following.

- The highlighted part includes a text of the form : `<bb dd>` where dd is a number. The prefix “bb” is an abbreviation for “basic block”.
- There are statements which begin with the label `<bb dd> :`, the statements that follow this label are the contents of this block. For example, the contents of `<bb 3>` are as shown below.

`<bb 3> :`

`35 : d = a * 15;`

`36 : if (i <= 4) goto <bb 4>; else goto <bb 5>;`

- We have inserted a label number to each gimple code, similar to what we add in our SDTS. This is done to refer to each gimple code by their label. Gimple code which are already labeled are not added a numeric label.
- The nodes (also called as basic blocks) of a cfg are drawn as rectangles. The notation “48:51” denotes that all gimple code statements with labels 48 through 51 are part of the basic block..

cfg constructed by gcc in textual form	cfg constructed by gcc displayed in graphical form
<pre> main () { // declaration statements removed <bb 2> : 20 : a = 2; 21 : b = 3; 22 : c = 40; 23 : x[0] = 10; 24 : x[1] = 20; 25 : x[2] = 30; 26 : x[3] = 40; 27 : x[4] = 50; 28 : x[5] = 60; 29 : x[6] = 70; 30 : x[7] = 80; 31 : x[8] = 90; 32 : x[9] = 100; 33 : i = 1; 34 : goto <bb 7>; <bb 3> : 35 : d = a * 15; 36 : if (i <= 4) goto <bb 4>; else goto <bb 5>; <bb 4> : 37 : _1 = i + -1; 38 : _2 = x[_1]; 39 : _3 = d + _2; 40 : x[i] = _3; 41 : b = a * c; 42 : goto <bb 6>; <bb 5> : 43 : b = a * c; 44 : d = a * c; 45 : c = b + d; <bb 6> : 46 : i = i + 1; <bb 7> : 47 : if (i <= 10) goto <bb 3>; else goto <bb 8>; <bb 8> : 48 : _4 = x[5]; 49 : printf (" a = %d b = %d d= %d x[5] = %d \n", a, b, d, _4); 50 : D.2329 = 0; 51 : return D.2329; } </pre>	<pre> graph TD bb2["bb2 start node 20 : 34 34 : goto bb7"] --> bb7["bb7 47 : 47"] bb7 -- T --> bb3["bb3 35 : 36"] bb7 -- F --> bb8["bb8 48 : 51 exit node"] bb3 -- T --> bb4["bb4 37 : 42 42 : goto bb6"] bb3 -- F --> bb5["bb5 43 : 45"] bb4 --> bb6["bb6 46 : 46"] bb5 --> bb6 bb6 --> bb7 </pre>

5. The last code in a basic block, $\langle \text{bb } j \rangle$, may be an assignment, a conditional or an unconditional transfer of control.
- If the last gimple code in block, $\langle \text{bb } i \rangle$, is an assignment, then the next gimple code must be the target of a goto statement, since otherwise the next code would be a part of block $\langle \text{bb } i \rangle$ itself, for instance $\langle \text{bb } 5 \rangle$ in our example.
 - If the last statement in block, $\langle \text{bb } i \rangle$, is an unconditional “goto $\langle \text{bb } j \rangle$,” then we draw an edge from block $\langle \text{bb } i \rangle$ to block $\langle \text{bb } j \rangle$. For instance $\langle \text{bb } 2 \rangle$ and $\langle \text{bb } 4 \rangle$.
 - When the last code in a block, $\langle \text{bb } i \rangle$, is a conditional goto, such as “if a relop b then goto $\langle \text{bb } j \rangle$ else goto $\langle \text{bb } k \rangle$,” then we draw two edges leaving from $\langle \text{bb } i \rangle$, one edge labeled with T (or True) that connects $\langle \text{bb } i \rangle$ and $\langle \text{bb } j \rangle$, while the other edge labeled F (or False) joins $\langle \text{bb } i \rangle$ to $\langle \text{bb } k \rangle$. We have used this in the blocks, $\langle \text{bb } 3 \rangle$ and $\langle \text{bb } 7 \rangle$.
 - The start node of a control flow graph is assumed to be unique, it is the block that contains the first gimple code, $\langle \text{bb } 2 \rangle$ in the example. The start node has the property that it has no predecessors, that is, $\text{predecessor}(\text{start}) = \Phi$. Predecessor of a node y is defined by, $\text{predecessor}(y) = \{x, \text{ where } x \text{ is a node, and } (x, y) \text{ is an edge in the cfg}\}$.
 - The exit nodes of a cfg are all the basic blocks that contain a “return” gimple code. In our example, block $\langle \text{bb } 8 \rangle$ is the unique exit node. The start node has the property that it has no successors, that is, $\text{successor}(\text{exit}) = \Phi$. Successor of a node x is defined by, $\text{successor}(x) = \{y, \text{ where } y \text{ is a node, and } (x, y) \text{ is an edge in the cfg}\}$. A cfg may have many exit nodes.

Having seen the structure of a cfg in textual and graphical form, an algorithm that directly constructs a cfg from intermediate code is given below.

2. Construction of Basic Blocks

Algorithm : Partition intermediate code into basic blocks and construct a control flow graph

i/p: A sequence of three-address statements after semantic analysis

o/p : Control flow graph

Method outline :

1. Identify the set of headers (also called as leaders), which are the first statements of basic blocks. Headers of basic blocks are identified as follows.

- The first statement is a header.
- Any statement that is the target of a conditional or unconditional goto is a header.
- Any statement that immediately follows a goto or conditional goto statement is a header.

2. Extract the body of each basic block. For each header, its basic block consists of the header and all statements up to but not including the next header or the end of the program.
3. Insert the edges between the basic blocks. Insert edges from the basic block to its successor blocks.

We illustrate the application of the algorithm on our example gimple code.

Gimple intermediate code with headers highlighted	Basic Blocks with header and other statements
<pre> main ()main () { // declaration statements removed 20 : a = 2; ## first gimple code 21 : b = 3; 22 : c = 40; 23 : x[0] = 10; 24 : x[1] = 20; 25 : x[2] = 30; 26 : x[3] = 40; 27 : x[4] = 50; 28 : x[5] = 60; 29 : x[6] = 70; 30 : x[7] = 80; 31 : x[8] = 90; 32 : x[9] = 100; 33 : i = 1; 34 : goto <D.2323>; <D.2322> : ## follows goto in code 34; also target of 47 35 : d = a * 15; 36 : if (i <= 4) goto <D.2326>; else goto <D.2327>; <D.2326> : ## target of code 36 also follows 36 37 : _1 = i + -1; 38 : _2 = x[_1]; 39 : _3 = d + _2; 40 : x[i] = _3; 41 : b = a * c; 42 : goto <D.2328>; <D.2327> : ## target of code 36 43 : b = a * c; 44 : d = a * c; 45 : c = b + d; <D.2328> : ## target of code 42 46 : i = i + 1; <D.2323> : ## target of code 34 47: if (i <= 10) goto <D.2322>; else goto <D.2324>; <D.2324> : target of code 47 and also follows 47 48 : _4 = x[5]; 49 : printf (" a = %d b = %d d= %d x[5] = %d\n", a, b, d, _4); 50 : D.2329 = 0; </pre>	Extract all statements from a header till just before the next header. Comments are inserted for readability and start with ##. basic block 1 : named as <bb 1> 20 : a = 2; ## first gimple code 21 : b = 3; 22 : c = 40; 23 : x[0] = 10; 24 : x[1] = 20; 25 : x[2] = 30; 26 : x[3] = 40; 27 : x[4] = 50; 28 : x[5] = 60; 29 : x[6] = 70; 30 : x[7] = 80; 31 : x[8] = 90; 32 : x[9] = 100; 33 : i = 1; 34 : goto <D.2323>; ## D.2323 starts new <bb 2> basic block 3 : named as <bb 3> <D.2322> : 35 : d = a * 15; 36 : if (i <= 4) goto <D.2326>; else goto <D.2327>; ## D.2326 is <bb 4> ## D.2327 is <bb 5> basic block 4 : named as <bb 4> <D.2326> : 37 : _1 = i + -1; 38 : _2 = x[_1]; 39 : _3 = d + _2; 40 : x[i] = _3; 41 : b = a * c; 42 : goto <D.2328>; ## starts <bb 6> basic block 5 : named as <bb 5> <D.2327> : 43 : b = a * c; 44 : d = a * c; 45 : c = b + d;

<pre> 51 : return D.2329; } </pre>	<pre> basic block 6 : named as <bb 6> <D.2328> : 46 : i = i + 1; basic block 2 : named as <bb 2> <D.2323> : 47: if (i <= 10) goto <D.2322>; else goto <D.2324>; ## D.2322 exists as is <bb 3> ## D.2324 is new block <bb 7> basic block 7 : named as <bb 7> <D.2324> : 48 : _4 = x[5]; 49 : printf (" a = %d b = %d d= %d x[5] = %d \n", a, b, d, _4); 50 : D.2329 = 0; 51 : return D.2329; </pre>
------------------------------------	---

The basic blocks have been numbered in the order they are encountered in the algorithm. The basic block numbers, the corresponding gimple label and the first code number assigned by us are summarized in the following table.

Basic block	Gimple Label	Target of Code No.	Header Code No.	Basic block contents : Start No : Last No
<bb1 >	---	---	20	20 : 34
<bb2 >	D.2323	34	47	47 : 47
<bb3 >	D.2322	47	35	35 : 36
<bb4 >	D.2326	36	37	37 : 42
<bb5 >	D.2327	36	43	43 : 45
<bb6 >	D.2328	42	46	46 : 46
<bb7 >	D.2324	47	48	48 : 51

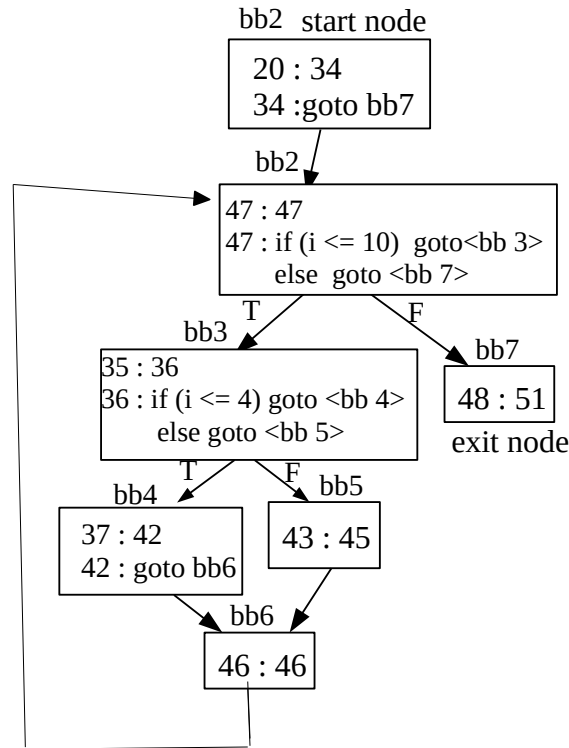
A final pass over the basic blocks and their contents as given in column 2 of the table above, generates the cfg in terms of the basic blocks and their connected edges. The final cfg is shown below.

The control transfer statements, both conditional and unconditional, have been added as the last statements of basic blocks wherever they are relevant, for readability only.

Properties of a basic block :

- All statements in a basic block are executed sequentially.
- Transfer from other basic blocks are only permitted to the header of the block, that is the first statement in the block.
- Transfer from within a basic block is not permitted, except at the last statement.

A basic block is the largest sequence of statements that can be executed sequentially once the control reaches the header of the block.



Note that the cfg constructed by us using the algorithm and the cfg reported by gcc are identical in content except for the numbering of the basic blocks (nodes).

3. Use of Basic Blocks / control flow graph in Optimization: The control flow graph exhibits the control structure in the program explicitly. Loops in the control flow graph depict parts of the code that are expected to be executed repeatedly. Therefore improvement in the contents of basic blocks that constitute a loop at compile time should result in significant savings at execution time.

- The cfg is useful for another significant optimization, known as detection of unreachable code. Unreachable code comprises of one or more basic blocks which are not reachable from the start node of the cfg. A basic block x , other than start node, that is $x \neq \text{start}$, is unreachable from start if $\text{predecessor}(x) = \Phi$. Finding all unreachable nodes in a cfg is a simple graph theoretic problem in directed graphs and can be determined. All such nodes can be safely removed from the cfg without changing the semantics of the underlying program. This optimization is named as **elimination of unreachable code**.
- Detecting all loops in a cfg**, including nested loops, is also a well known graph theoretic problem in directed graphs. One may use DFS to detect and extract the loops in a cfg in terms of the set of basic blocks that constitute the loop. For the cfg of our example, there is exactly one loop, described by the set of basic blocks, $\{\langle \text{bb } 2 \rangle, \langle \text{bb } 3 \rangle, \langle \text{bb } 4 \rangle, \langle \text{bb } 5 \rangle, \langle \text{bb } 6 \rangle, \langle \text{bb } 2 \rangle\}$ which can only be entered through the block $\langle \text{bb } 2 \rangle$.

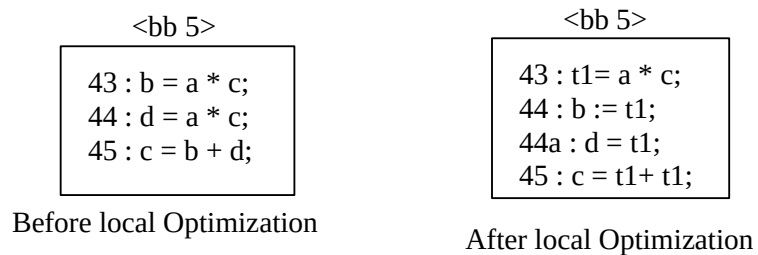
The main purpose of the cfg is to perform optimizations on the intermediate code in order to improve its efficiency. Analysis of the **structure of the cfg, is known as control flow analysis**. Control flow analysis is used in optimizations, popularly known as control flow transformations, that may result in addition / deletion of nodes or edges, but usually do not change the contents of the basic blocks. Removal of unreachable code, detection of loops, are some of the examples of control flow analysis and transformations.

Analysis of the **contents of basic blocks is known as data flow analysis**. If the analysis is confined to within a basic block, also called as intra-basic block, then it is called as local data flow analysis. On the contrary, when the contents of all basic blocks are analysed, for some improvement, then the resulting analysis is known as global data flow analysis.

Data Flow Optimizations are broadly divided into 2 classes.

1. Local Optimization: Improvements in the contents within a basic block are usually termed as local optimization. The scope of a local optimization is a basic block only. The analysis required for local optimization is limited and therefore such optimizations can be performed efficiently with $O(\text{number of statements in block})$ effort. Examples could be eliminating redundant computations, other optimizations such as constant folding, constant propagation, etc., that are described later.

An instance of local code optimization exists in basic block, <bb 5> in our example. Result of removing the redundant computation is shown below. Whether the new assignments to b and d are used in the rest of the cfg requires global data analysis and will not be handled by the local optimization.



2. Global Optimization : Improvements in the contents of basic blocks by examining the entire cfg including all its basic blocks, fall under the scope of global optimization. Such computations are expensive since they require analysing the entire control graph and are usually $O(n^2)$, where n is the number of nodes in the cfg. Examples are given later. While they are expensive to implement the savings are significant when opportunities for such optimizations are detected.

Illustration of Optimization through Example

The approach we take is to illustrate the capability of the optimization phase through examples. All the optimizations shall be performed on the source code itself, though we know that in reality they are performed by the compiler on the intermediate code (IC). The reason for the choice of source code is to communicate the intent and spirit of the optimizations without using the volume and verbosity of IC.

Consider the following C program which has a main() function. An integer array and few scalar variables are defined here. There is a for-loop which has an if-then-else statement within the body of the for-loop. Another if-then-else statement follows the for-loop.

```
int main()
{ int a = 2, b = 3, c = 40, d, i, j;
  int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
  for ( i = 1; i < 11; i+=2 )
  { if ( i%2)
    { d = a * b; x[i] = x[i-1] + d; c = b*100; }
    else
    {d = a * a; x[(i+d)%10] = x[i]+x[i-1];}
  };
  if (a < 2) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5;
  else
    for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]);
  return 0;
}
```

The following exercise may be done by the readers to independently examine the effect optimizations versus no optimization performed by gcc on the program above. Let's name the file as "prog1.c". The switch, "-fverbose-asm" asks the compiler to add annotations to the generated assembly, which is generally more readable.

- generate assembly code without optimization

```
$ gcc -S -fverbose-asm prog1.c
```

```
$ mv prog1.s prog1_unopt.s
```

- generate assembly code at optimization level 2

```
$ gcc -S -O2 -fverbose-asm prog1.c
```

- Compare the two assembly codes, "prog1_unopt.s" and "prog1.s" and make your own observations.

Optimization1 : Can a compiler perform some computations at compile time and save the cost incurred at run time ? Let us explore situations in prog1.c which permit such computations.

Source Code	Opportunities for improvement
<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; for (i = 1; i < 11; i+=2) { if (i%2) { d = a * b; x[i] = x[i-1] + d; c = b*100; } else { d = a * a; x[(i+d)%10] = x[i]+x[i-1];} }; if (a < 2) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>	Note that a and b are both initialized to 2 and 3 and hence compiler may evaluate d at compile time Same situation for c = b * 100 same for d = a * a also true for a < 2, since a is known

Performing these compile time computations produces the program in column 2

<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; for (i = 1; i < 11; i+=2) { if (i%2) { d = a * b; x[i] = x[i-1] + d; c = b*100; } else { d = a * a; x[(i+d)%10] = x[i]+x[i-1];} }; if (a < 2) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>	<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; for (i = 1; i < 11; i+=2) { if (i%2) { d = 6; x[i] = x[i-1] + d; c = 300;} else { d = 4; x[(i+d)%10] = x[i]+x[i-1];} }; if (false) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>
--	--

This optimization is known as **Constant Folding**.

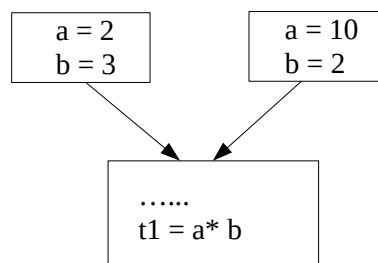
Statement of the Constant Folding :

An intermediate code statement of the form : “a := b op c” is to be replaced by the constant values of its operands, b and c, say v1 and v2 respectively, to the new code “a := v3” where v3 is the result of

$v1 * v2$ that is by computed at compile time itself.

Benefits : An operation is saved at run time, as the computed value can be directly used. The savings are more for complex operations, such as $*$, $/$, $\%$, etc as they consume more cycle time during execution.

Constraints : This optimization requires that the operand values reaching the statement are unique. A counterexample where this optimization would change the semantics of the original program is shown below. It is not possible to fold the computation, “ $t1 = a * b$ ”, even though both the operands are constants in both the paths reaching the point, because the semantics will not be the same after folding.



Analysis : Compiler has to perform an analysis of the IC to determine at the program point of interest, say p , the definitions of all its operands that reach the point p . The analysis performed by the compiler is known as “Reaching Definitions Analysis” and is beyond the scope of this course. However, once the analysis has been performed by the compiler, it can easily check if the operands have a single definition of both the operands, and in this case can safely perform constant folding.

Home work : Analyse and report whether all the 4 instances of constant folding in the example given are safe.

Optimization 2 : Once constant folding has been successfully done for some computations, can this lead to more optimizations at compile time ? Let us explore situations in prog2.c, obtained after folding on prog1.c

The use of constants in expressions, highlighted below, are possibilities of optimization where the constant values are directly substituted instead of the variables.

After constant folding : prog2.c	Prog3.c after constant propagation
<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; for (i = 1; i < 11; i+=2) { if (i%2) { d = 6; x[i] = x[i-1] + d; c = 300;} else {d = 4; x[(i+d)%10] = x[i]+x[i-1];} }; if (false) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>	<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; for (i = 1; i < 11; i+=2) { if (i%2) { d = 6; x[i] = x[i-1] + 6; // propagate const value of d c = 300; } else {d = 4; x[(i+4)%10] = x[i]+x[i-1];} // propagate value of d }; if (false) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>

This optimization is known as **Constant Propagation**. The main objective is to propagate constants detected at compile time for further optimizations including constant folding.

Statement of Constant Propagation :

An intermediate code statement of the form : “a := b op c” is to be replaced by the value of its operands, b or c, if one of them are found out to be constant at compile time. For example, if b has the value v1, then IC is changed to IC “a := v1 op c”.

Benefits : An memory access is saved at run time at the least, this optimization may also expose more instances of constant folding.

Constraints : This optimization has the same constraints as that mentioned for constant folding, that is the operand value(s) reaching the statement is (are) unique. Similar counterexample can be created for this optimization also.

Analysis : The “Reaching Definitions Analysis” mentioned earlier, can also be used to perform this optimization safely.

Home work : Analyse and report whether all the 2 instances of constant propagation in the example given are safe.

Optimization 3 : While the two optimizations discussed earlier are useful, larger benefits accrue if some computations within loops can be safely moved of a loop to a place outside the loop. This optimization is known as **Loop Invariant Code Motion**. This optimization involves two tasks,

- detect a computation that is invariant in the loop it is placed, that is, it does not depend on the loop surrounding it and has exactly the same value on every iteration of the loop, and
- find a place to move the loop invariant code outside the enclosing loop.

This optimization also, like all others that we discuss, must not change the semantics of the IC. For the modified program, prog3.c, three opportunities are highlighted using 3 different colors in 2nd column of the following table. These computations have been pulled out the for-loop and placed just before the loop.

After constant folding & propagation : prog3.c	Situations for optimization : prog4.c
<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; for (i = 1; i < 11; i+=2) { if (i%2) { d = 6; x[i] = x[i-1] + 6; c = 300; } else { d = 4; x[(i+4)%10]= x[i]+x[i-1];} }; if (false) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>	<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; d = 6; c = 300; d = 4; for (i = 1; i < 11; i+=2) { if (i%2) { // d = 6; loop invariant x[i] = x[i-1] + 6; // c = 300; loop invariant } else { // d = 4; loop invariant x[(i+4)%10] = x[i]+x[i-1];} }; if (false) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>

Statement of Loop invariant Code Motion : Given a loop L of the cfg and a computation or IC of the form, “ a = b op c” at some point p in some basic block of L, determine if “ a = b op c” is an loop invariant of L. An IC, “ a = b op c”, is a loop invariant of a loop L, if all the definitions of the operands b and c are placed outside L. Then this code can be safely moved out from L to a suitable predecessor / successor of L.

Benefits : Movement of loop invariant code outside a loop at compile time reduces the computation effort of m*x (where m is the number of iterations of loop at run time, and x is one time cost of

execution of the code), to x.

Constraints : All operands involved in a loop invariant code must have their definitions from outside the given loop.

Analysis : Requires detection of a loop L in cfg, followed by analysis of all basic blocks in L.

Optimization 4 : Dead Code Elimination is yet another optimization performed by a compiler. As the name suggest, code that are not used in the program, after their definition at some point p, may be safely removed without changing the semantics of the original program. This optimization involves two tasks,

- detect a computation, defined at program point p, that is no longer used on any path starting from p to the rest of the program till its exit point.
- Change the IC by removing the corresponding IC.

This optimization also, like all others that we discuss, must not change the semantics of the IC. For the modified program, prog4.c, three definitions, that are highlighted are dead from their point of definition and hence can be safely removed.

After constant folding & propagation : prog4.c	Situations for optimization : prog5.c
<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; d = 6; c = 300; d = 4; for (i = 1; i < 11; i+=2) { if (i%2) { x[i] = x[i-1] + 6; } else { x[(i+4)%10] = x[i]+x[i-1]; } }; if (false) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>	<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; // d = 6; dead variable // c = 300; dead variable // d = 4; dead variable for (i = 1; i < 11; i+=2) { if (i%2) { x[i] = x[i-1] + 6; } else { x[(i+4)%10] = x[i]+x[i-1]; } }; if (false) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>

Statement of Dead Code Elimination : IC statements that are **not used** after their definition at point p on any path in the cfg starting from p to the rest of the cfg till its exit block, can be safely removed.

Benefits : Saves memory and also reduces execution time.

Constraints : The variable defined in the code, needs to have at least one use on any path starting from the point of interest p , in order not to be declared as dead at p . Variables who have at least one use from the point of interest p are referred to as live variables at p . A variable that is not live at a point p becomes a candidate for dead code.

Analysis : Live variable Analysis is performed by the compiler to detect all variables that are live (or have some use at that point or later) at every program point in the program. Variables that are not found to be live, are dead, and can be safely eliminated.

Optimization 5 : Elimination of Unreachable Code

This optimization involves elimination of unreachable code from IC. In prog5.c, the conditional “if(false)” is always false, so that control never reaches the for-loop in the then part of the conditional. These statements may therefore be safely removed without changing the underlying semantics. Code fragments that are unreachable due to evaluation of conditional expressions in control flow constructs, or statements that immediately follow an unconditional transfer of control, such as “goto”, “return”, “break”, etc are examples of such code.

After Dead Code Elimination : prog5.c	Situations for optimization : prog6.c
<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; for (i = 1; i < 11; i+=2) { if (i%2) { x[i] = x[i-1] + 6; } else { x[(i+4)%10] = x[i]+x[i-1];} }; if (false) for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>	<pre>int main() { int a = 2, b = 3, c = 40, d, i, j; int x[10]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100}; for (i = 1; i < 11; i+=2) { if (i%2) { x[i] = x[i-1] + 6; } else { x[(i+4)%10] = x[i]+x[i-1];} }; // if (false) // always false unreachable code // for (j = 1; j < 11; j++) x[(j+5)%10]=x[j]+5; //else for (j = 0; j < 10; j++) printf(" x[%d] = %d \n", j, x[j]); return 0; }</pre>

Statement of Unreachable Code Elimination : Statements that are unreachable from the start node of the cfg are known as “unreachable code”. This may result either from control flow analysis, or due to the application of optimizations which lead to some conditional expression to have a constant value, True or False, at compile time.

Benefits : Saves both memory and execution time. Also identifies unintentional potential bugs in the design / code.

Constraints : Correctness depends on the the compile time evaluations done under various optimizations by the compiler which has resulted in some code marked as unreachable.

Analysis : Control flow analysis is the key analysis performed by the compiler to detect code that are found to be unreachable in the cfg. Global data flow analysis is required to determine code that reachable from the start block but will never be executed at run time because some conditional expression has been analysed and found to have acquired a constant value at compile time.

In summary, six different optimizations were applied above in some order to convert the given source program to the final form shown in the following table. Readers are urged to compile and run all the six different versions of the program, from prog1.c to prog6.c. Verify whether all the programs produce the same output.

Homework : As a take home exercise, students are urged to find if s/he can define an optimization, other than those mentioned above, and apply to the final version, prog6.c ,in order to make it even more efficient.

Give a suitable name to the optimization, and describe it in terms of the components : statement of the optimization, benefits, constraints and analysis required. Provide an example of application of your optimization giving the program code before and after the optimization.

End of Document